



Universidad Tecnológica Nacional
Facultad Regional Mendoza
Ingeniería en Sistemas de Información

TRABAJO PRÁCTICO INTEGRADOR

Estructuras de Datos, Contrato Equals/HashCode y
Complejidad Algorítmica

Cátedra: Desarrollo de Software

Profesores:

- Alberto Cortez
- Magni Gerardo
- Agustín Cassani

Comisión: 3K10

Turno: Noche

Ciclo Lectivo: 2026

Integrantes	Legajo
Rodríguez Quiñonez, Jessica Magaly	48913
Sosa, Renzo	50173
Sosa Avila, Florencia Natasha	50716

Introducción.....	2
Objetivos.....	3
Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria.....	4
Responder y Justificar:.....	5
Resolución del Ejercicio 1: Diagnóstico de Bug Silencioso y Análisis de Memoria.....	5
1. Traza del Estado Interno del HashSet.....	5
2. Explicación del Cajón (Bucket).....	6
3. Fuga de Memoria (Memory Leak).....	6
4. Refactorización Obligatoria.....	6
Consigna 2: Selección y Diseño de Arquitectura de Colecciones.....	8
Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$).....	9
Resolución del Ejercicio 3: Optimización Algorítmica.....	9
1. Refactorización a $O(n)$	9
2. Análisis de Colisiones en HashMap.....	10
Consigna 4: Caso Teórico de Borde y Colisiones.....	11
Resolución del Ejercicio 4: Caso Teórico de Borde y Colisiones.....	11
1. Inserción en HashSet y funcionamiento de hashCode() y equals().....	11
2. Rendimiento en el peor escenario (Acumulación de colisiones).....	12
Conclusión.....	13

Introducción

En el desarrollo de software backend, la elección y manipulación correcta de las estructuras de datos es un pilar fundamental para garantizar el rendimiento, la escalabilidad y la estabilidad de las aplicaciones. El ecosistema de Java, a través del paquete `java.util`, provee un conjunto robusto de colecciones; sin embargo, su uso eficiente requiere una comprensión profunda de sus mecanismos internos.

El presente informe técnico aborda el análisis, diagnóstico y optimización de implementaciones basadas en estructuras de tipo Hash (`HashSet`, `HashMap`). A través de la resolución de casos prácticos, se explorará el impacto que tiene el contrato entre los métodos `equals()` y `hashCode()`, los riesgos asociados a la mutabilidad de los objetos almacenados y las estrategias algorítmicas necesarias para optimizar el procesamiento de datos, garantizando así un código seguro para entornos de producción.

Objetivos

- **Comprender la arquitectura interna de las colecciones Hash:** Analizar el funcionamiento de los *buckets*, la función de dispersión y la resolución de colisiones mediante encadenamiento separado y árboles balanceados.
- **Diagnosticar vulnerabilidades de memoria:** Identificar cómo la mutabilidad de los atributos que definen la identidad de un objeto rompe el contrato de las colecciones, generando falsos negativos en búsquedas y fugas de memoria (*Memory Leaks*).
- **Aplicar principios de diseño seguro:** Implementar soluciones arquitectónicas basadas en inmutabilidad y en la correcta definición de los criterios de identidad comercial para proteger la integridad de los datos.
- **Optimizar la complejidad algorítmica:** Refactorizar procesos iterativos ineficientes de orden $O(n^2)$, trasladándolos a soluciones de orden lineal $O(n)$ mediante el uso estratégico de mapas y recorridos secuenciales.

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Un equipo de desarrollo junior implementó el siguiente sistema para gestionar el acceso de empleados a un edificio mediante tarjetas de identificación mutable. El código no lanza excepciones en tiempo de ejecución, pero presenta comportamientos anómalos en producción.

```
import java.util.*;
class TarjetaAcceso {
    private String codigo;
    private int nivelAcceso;
    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }
    public void setNivelAcceso(int nivelAcceso) {
        this.nivelAcceso = nivelAcceso;
    }
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        return nivelAcceso == tarjeta.nivelAcceso && Objects.equals(codigo,
            tarjeta.codigo);
    }
    @Override
    public int hashCode() {
        return Objects.hash(codigo, nivelAcceso);
    }
}
```

```
public static void main(String[] args) {
    Set<TarjetaAcceso> tarjetasValidas = new HashSet<>();[cite: 3]
    TarjetaAcceso t1 = new TarjetaAcceso("ACC-101", 2);
    tarjetasValidas.add(t1);
    // Modificación posterior a la inserción
    t1.setNivelAcceso(5);
    System.out.println("¿Tarjeta válida en sistema?: " +
        tarjetasValidas.contains(t1));
    System.out.println("Cantidad de tarjetas registradas: " +
        tarjetasValidas.size());
}
```

Responder y Justificar:

1. Traza del Estado Interno: Explique paso a paso qué ocurre dentro de la estructura HashSet:
 - Al ejecutar `tarjetasValidas.add(t1)`.
 - Al ejecutar `t1.setNivelAcceso(5)`.
 - Al ejecutar `tarjetasValidas.contains(t1)`.
2. Explicación del Cajón (Bucket): ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer `contains()`?
3. Fuga de Memoria (Memory Leak): ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?
4. Refactorización Obligatoria: Aplique dos soluciones arquitectónicas distintas para corregir el
5. problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

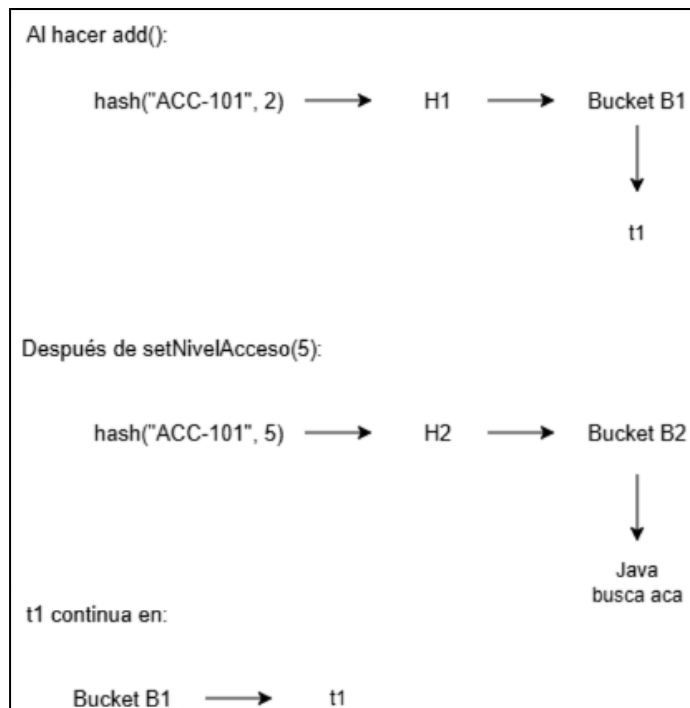
Resolución del Ejercicio 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

1. Traza del Estado Interno del HashSet

El problema radica en cómo la estructura `HashSet` gestiona la ubicación de los objetos en memoria basándose en su estado al momento de la inserción.

- **Al ejecutar `tarjetasValidas.add(t1)`:** Se instancia `t1` con el código `"ACC-101"` y nivel 2. El método `add()` invoca automáticamente a `t1.hashCode()`, el cual calcula un valor entero basado en ambos atributos (`Objects.hash("ACC-101", 2)`). Con este valor, Java determina en qué *bucket* (cajón o índice del arreglo interno) se guardará la referencia a este objeto.
- **Al ejecutar `t1.setNivelAcceso(5)`:** Se modifica el estado interno del objeto (su nivel pasa de 2 a 5). Sin embargo, el `HashSet` **no se reorganiza automáticamente**. El objeto continúa físicamente en el *bucket* donde fue colocado originalmente, pero ahora su estado ya no coincide con el hash de ese *bucket*.
- **Al ejecutar `tarjetasValidas.contains(t1)`:** El método `contains()` necesita buscar el objeto. Para hacerlo de forma eficiente ($O(1)$), calcula nuevamente el `hashCode()` de `t1`. Como el nivel ahora es 5, la función matemática devuelve un hash completamente distinto al original. Java va a buscar el objeto a este **nuevo bucket**, lo encuentra vacío (o con otros objetos) y, al no hallarlo, retorna `false`. El objeto original sigue estando en la colección, pero se ha vuelto "invisible" para los métodos de búsqueda.

2. Explicación del Cajón (Bucket)



Al momento de la inserción, el objeto generó un hash (por ejemplo, el hash **X**) que lo ubicó en el **Cajón A**. Cuando mutamos el objeto, su nuevo estado genera un hash diferente (hash **Y**). Al ejecutar `contains()`, Java busca el objeto en el **Cajón B** (correspondiente al hash **Y**). Como el objeto nunca fue movido del Cajón A, Java asume erróneamente que no existe en la colección.

3. Fuga de Memoria (Memory Leak)

Este patrón genera una fuga de memoria silenciosa en un servidor 24/7 porque los objetos modificados quedan "huérfanos" dentro de la estructura. Si el sistema continuamente agrega objetos al `HashSet` y luego los muta, estos se vuelven inalcanzables mediante `contains()` o `remove()`. Sin embargo, como el `HashSet` sigue manteniendo una referencia fuerte hacia ellos en sus *buckets* originales, el **Garbage Collector (Recolector de Basura)** de Java no puede liberar esa memoria. Con el paso del tiempo y las ejecuciones repetidas, la colección crecerá infinitamente hasta agotar la memoria disponible de la JVM (`OutOfMemoryError`).

4. Refactorización Obligatoria

Existen dos enfoques arquitectónicos válidos para solucionar este problema:

Opción A - Inmutabilidad de la Entidad (Value Object) Se bloquea la posibilidad de mutar el objeto después de ser creado. Si el hash nunca cambia, el objeto nunca se "pierde" en el Set. Si se requiere un cambio, se debe instanciar un objeto nuevo.

```
final class TarjetaAcceso {
    private final String codigo;
    private final int nivelAcceso;

    public TarjetaAcceso(String codigo, int nivelAcceso) {
        this.codigo = codigo;
        this.nivelAcceso = nivelAcceso;
    }

    // Sin setters. Si cambia el nivel, se crea una tarjeta nueva:
    public TarjetaAcceso conNivelAcceso(int nuevoNivel) {
        return new TarjetaAcceso(this.codigo, nuevoNivel);
    }

    // equals() y hashCode() se mantienen evaluando ambos atributos
}
```

Opción B - Ajuste del Criterio de Identidad (Recomendada para Entidades) Se excluye el campo mutable (`nivelAcceso`) del cálculo de identidad. Un objeto se define únicamente por su clave de negocio inmutable (el `codigo`). Así, aunque el nivel cambie, su hash seguirá siendo exactamente el mismo.

```
class TarjetaAcceso {
    // ... constructores y atributos ...

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        TarjetaAcceso tarjeta = (TarjetaAcceso) o;
        // Solo evaluamos el campo inmutable
        return Objects.equals(codigo, tarjeta.codigo);
    }

    @Override
    public int hashCode() {
        // Solo hasheamos el código, garantizando que nunca cambie de
        bucket
        return Objects.hash(codigo);
    }
}
```


Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Diseñe el backend de un módulo para una plataforma de E-Commerce. Debe elegir la interfaz y la implementación concreta del Java Collections Framework (java.util) más eficiente para cada una de las siguientes tres funcionalidades:

Requerimiento 1: Sistema de Auditoría de Transacciones

Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles de registros por segundo al final de la colección y recorrerlos secuencialmente.

Requerimiento 2: Catálogo de Ofertas Relámpago

Mantener los productos ordenados automáticamente por precio de menor a mayor. Debe permitir consultar eficientemente el producto más barato y el más caro.

Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos

Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre 500,000 usuarios conectados en simultáneo.

Requerimientos	Interfaz Seleccionada (List, Set, Map, Queue)	Implementación Concreta (ArrayList, TreeSet, HashMap, etc)	Complejidad Búsqueda	Complejidad Inserción	Justificación técnica de la elección
Auditoría	List	ArrayList	$O(n)$	$O(1)$	Solo se inserta al final y se recorre secuencialmente; no hace falta orden ni búsqueda por clave. ArrayList tiene mejor localidad de memoria (cache-friendly) que LinkedList para recorridos.
Ofertas	SortedSet (Set)	TreeSet	$O(\log n)$	$O(\log n)$	Internamente usa un árbol rojo-negro balanceado que mantiene los elementos siempre ordenados por precio. first() y last() dan acceso directo al más barato/caro en $O(\log n)$ (prácticamente $O(1)$ en la práctica)
Sesiones	Map	HashMap	$O(1)$	$O(1)$	Se busca por clave exacta (el Token), sin necesidad de orden. La tabla hash da acceso casi instantáneo promedio, ideal para 500.000 usuarios simultáneos

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'r').

Resolución del Ejercicio 3: Optimización Algorítmica

1. Refactorización a $O(n)$

Para lograr la optimización requerida, se utiliza la estructura `HashMap`, separando la lógica en dos recorridos lineales independientes. A continuación se presenta el código fuente detallado en la imagen de referencia ([image_750c85.png](#)):

```
import java.util.HashMap;
import java.util.Map;

public class Ejercicio3 {

    public static Character primerCaracterNoRepetido(String texto) {
        if (texto == null || texto.isEmpty()) {
            return null;
        }

        Map<Character, Integer> conteo = new HashMap<>();

        // Primer recorrido  $O(n)$ : cuenta la frecuencia de cada carácter
        for (int i = 0; i < texto.length(); i++) {
            char c = texto.charAt(i);
            conteo.put(c, conteo.getOrDefault(c, 0) + 1);
        }

        // Segundo recorrido  $O(n)$ : busca el primer carácter con
        // frecuencia igual a 1
        for (int i = 0; i < texto.length(); i++) {
            char c = texto.charAt(i);
            if (conteo.get(c) == 1) {
                return c;
            }
        }

        return null;
    }
}
```

- **Complejidad Temporal:** El algoritmo es $O(n)$, ya que realiza dos iteraciones independientes sobre el texto de longitud n y las operaciones de inserción/búsqueda en el `HashMap` toman un tiempo promedio constante de $O(1)$.

- **Complejidad Espacial:** Es $O(k)$, donde k es el número de caracteres únicos. En el peor de los casos, este valor está acotado por el tamaño total del alfabeto utilizado, lo que lo hace muy eficiente en memoria.

2. Análisis de Colisiones en HashMap

Cuando dos claves distintas generan el mismo valor al invocar `hashCode()` (o se mapean al mismo índice de la tabla de *buckets* mediante la función de dispersión), ocurre lo que se conoce como una **colisión de hash**. Java resuelve este escenario mediante la técnica de **encadenamiento separado (Separate Chaining)**:

- **Estructura del Bucket:** Cada posición (*bucket*) del arreglo interno contiene una lista enlazada de nodos (`Node<K, V>`). Cada nodo almacena la clave, el valor, el hash calculado y la referencia al siguiente nodo.
- **Inserción con Colisión:** Si el *bucket* ya contiene elementos, se recorre la lista comparando las claves con el método `equals()`. Si la clave es diferente, el nuevo par clave-valor se añade como un nuevo nodo en esa misma posición sin sobrescribir el existente.
- **Recuperación (get):** Al consultar una clave, el `HashMap` localiza el *bucket* mediante el `hashCode()` y luego recorre los nodos comparando con `claveBuscada.equals(claveNodo)` para devolver el valor exacto correspondiente.
- **Optimización a Árbol Rojo-Negro (Java 8+):** Si el número de colisiones en un mismo *bucket* supera un umbral específico (por defecto 8 nodos, siempre que la tabla tenga una capacidad mínima de 64), la lista enlazada se transforma en un árbol binario balanceado (Red-Black Tree). Esto reduce drásticamente el costo de búsqueda en el peor caso, pasando de un tiempo lineal $O(n)$ a uno logarítmico $O(\log n)$.

Consigna 4: Caso Teórico de Borde y Colisiones

Considere las siguientes dos instancias de String:

- String s1 = "FB";
- String s2 = "Ea";

Sabiendo que `s1.hashCode() == s2.hashCode()` devuelve true (ambos dan el entero 2236):

1. Si guardamos ambas cadenas en un `HashSet<String>`, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos `hashCode()` y `equals()` internamente para determinar si s2 debe ingresarse o desecharse.
2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un `HashMap` en el peor escenario posible (de $O(1)$ a $O(n)$ o $O(\log n)$)?

Resolución del Ejercicio 4: Caso Teórico de Borde y Colisiones

1. Inserción en HashSet y funcionamiento de `hashCode()` y `equals()`

Si guardamos ambas cadenas ("FB" y "Ea") en un `HashSet`, la colección contendrá **2 elementos**. Ninguno será desechado.

Explicación del proceso interno: Internamente, un `HashSet` en Java está respaldado por un `HashMap`. Cuando intentamos insertar un elemento, Java ejecuta un proceso de verificación de dos pasos para asegurar que no haya duplicados, priorizando el rendimiento:

1. **Filtro rápido (`hashCode()`):** Al intentar ingresar `s2` ("Ea"), Java calcula su hash. Como `s1.hashCode() == s2.hashCode()` (ambos dan 2236), Java determina que `s2` debe ir al **mismo bucket** (índice del arreglo interno) donde ya está alojado `s1`. Esto es una **colisión de hash**.
2. **Verificación de identidad (`equals()`):** Que dos objetos compartan el mismo `hashCode` no significa que sean el mismo objeto, simplemente significa que "cayeron en la misma cubeta". Para desempatar, Java invoca el método `equals()` sobre los objetos que colisionaron. Compara `s1.equals(s2)` (es decir, `"FB".equals("Ea")`).
3. **Decisión final:** Como "FB" y "Ea" son cadenas con distinto contenido, el método `equals()` retorna `false`. Para el `HashSet`, esto confirma que **son objetos lógicamente distintos**. Por lo tanto, no desecha `s2`, sino que lo almacena en el mismo bucket que `s1` formando una estructura anidada (una lista enlazada).

Conclusión clave: El `hashCode()` indica *dónde* buscar o guardar, pero el `equals()` tiene la última palabra para determinar si dos elementos son realmente *iguales*.

2. Rendimiento en el peor escenario (Acumulación de colisiones)

En un escenario ideal, donde la función hash distribuye uniformemente las claves en los buckets, las operaciones básicas de un `HashMap` o `HashSet` (como `put`, `get`, `contains`) tienen una complejidad temporal constante de $O(1)$.

Sin embargo, en el **peor escenario teórico** (por ejemplo, si creáramos una clase con un mal diseño donde el método `hashCode()` siempre devolviera el mismo número, o si tuviéramos miles de cadenas diseñadas maliciosamente para colisionar), todos los elementos irían a parar al mismo bucket.

El impacto en el rendimiento se comporta de la siguiente manera, dependiendo de la versión de Java:

- **Degradación a $O(n)$ (Java 7 y anteriores):** Históricamente, las colisiones en un mismo bucket se manejaban exclusivamente encadenando los elementos en una **Lista Enlazada simple** (`LinkedList`). Si hay n elementos en un mismo bucket, para buscar o insertar uno nuevo, Java tenía que recorrer la lista nodo por nodo utilizando `equals()`. Esto degrada el rendimiento de búsqueda de $O(1)$ a $O(n)$ (tiempo lineal).
- **Mejora a $O(\log n)$ (A partir de Java 8):** Para mitigar este problema de rendimiento frente a colisiones masivas, Java introdujo una optimización. Cuando una lista enlazada dentro de un bucket supera un cierto umbral (actualmente 8 elementos, definido por la constante `TREEIFY_THRESHOLD`), la estructura de ese bucket se transforma automáticamente en un **Árbol Binario de Búsqueda Balanceado** (específicamente un árbol Rojo-Negro). Gracias a esta estructura de árbol, la búsqueda en el peor escenario no requiere recorrer todos los elementos uno por uno, reduciendo la complejidad temporal a $O(\log n)$, lo cual es exponencialmente más rápido que $O(n)$ cuando hay muchas colisiones.

Conclusión

La resolución de los casos expuestos demuestra que el rendimiento teórico constante $O(1)$ que prometen las estructuras basadas en tablas Hash no es una garantía implícita, sino el resultado de aplicar buenas prácticas de ingeniería de software.

Se evidenció que desconocer el funcionamiento interno del `HashSet` y la relación inquebrantable entre `equals()` y `hashCode()` puede derivar en fallas silenciosas severas, comprometiendo la memoria de la Máquina Virtual de Java (JVM). Asimismo, la refactorización algorítmica comprobó que el uso adecuado de un `HashMap` permite reducir drásticamente los tiempos de ejecución, transformando algoritmos cuadráticos ineficientes en procesos lineales óptimos.

En definitiva, dominar las colecciones en Java no se limita a conocer su sintaxis, sino a comprender su comportamiento en memoria, prever los escenarios de colisión y diseñar entidades robustas. Esto resulta indispensable para construir sistemas eficientes, escalables y libres de errores en entornos de alta demanda.